

Сортировка - упорядочение некоторого множества элементов

Сортировка включает

1. **Правило**, которое определяет упорядоченность пары - *ключ сортировки, правило сравнения*
2. **Порядок** сортировка (возрастание, убывание)
3. Непосредственно сортирующий **алгоритм**

Виды сортировки

1. Сортировка поиском (выбором)

Идея: Выбираем элемент на границе отсортированной части и не отсортированной

1	2	3	[	5	]	9	8	4
---	---	---	---	---	---	---	---	---

ищем локальный минимум и меняем местами элементы

1	2	3	4	[	9	]	8	5
---	---	---	---	---	---	---	---	---

```
size_t n = 10;
int a[n];

for (size_t i = 0; i < n - 1; i++) {
    size_t local_min_ind = i;

    // Ищем локальный минимум
    for (size_t j = i + 1; j < n; j++) {
        if (a[j] < a[local_min_ind]) {
```

```

        local_min_ind = j;
    }
}
// меняем местами элементы
std::swap(a[i], a[local_min_ind]);
}

```

2. Сортировка вставками

Идея: Берем элемент и смотрим где его можно разместить в левой упорядоченной части

Вариант 1

```

size_t n = 10;
int a[n];

for (size_t i = 1; i < n; i++) {
    // Берем элемент справа и двигаем влево
    for (size_t j = i; j > 0 && a[j] < a[j - 1]; j--)
    {
        // из-за свапа алгоритм неэффективен
        swap(a[j - 1], a[j]);
    }
}

```

Вариант 2

```

size_t n = 10;
int a[n];

// Более эффективен, за счет сохранения во временную
// переменную, а не свапа
for (size_t i = 1; i < n; i++) {

```

```

    int tmp = a[i];
    size_t j = i;

    while (j > 0 && tmp < a[j - 1]) {
        a[j] = a[j - 1];
        j--;
    }
    a[j] = tmp;
}

```

3. Сортировка пузырьком

Идея: Постепенно проходим по массиву и если элемент справа больше текущего, то меняем их местами. Самые большие элементы "всплывают" в конец массива

```

// переменная для того, чтобы понимать,
// есть ли смысл перебирать массив
bool f = true;
int j = 0;

while (f) {
    // надеемся, что все получится за один проход
    f = false;

    for (size_t i = 0; i + j + 1 < n; i++) {
        if (a[i] > a[i + 1]) {
            std::swap(a[i], a[i + 1]);
            // если хоть раз поменяли элементы,
            // то имеет смысл еще раз пройтись по
массиву
            f = true;
        }
    }
}

```

```
        j++;  
    }
```

4. Сортировка слиянием

Идея: Делим массив пополам, до тех пор пока не доделим до массивов из одного элемента, после начинаем процедуру слияния

```
// Вспомогательная функция,  
// которая сливает 2 массива в 1 упорядоченный.  
// Так как алгоритм рекурсивный,  
// то массивы на входе тоже упорядочены (!)  
void merge(int a[], int left, int mid, int right) {  
    int size = right - left + 1;  
    int* tmp = new int[size];  
  
    int i = left;  
    int j = mid + 1;  
    int k = 0;  
  
    // [... , left, ... mid, mid + 1, ... , right,  
    ...]  
  
    // пока оба массива еще не закончились  
    while (i <= mid && j <= right) {  
        // элемент поменьше записываем по k-му  
        // индексу  
        // в новый массив  
        if (a[i] <= a[j]) {  
            tmp[k++] = a[i++];  
        } else {  
            tmp[k++] = a[j++];  
        }  
    }  
}
```

```
// если элементы остались в каком-то из массивов,  
// то оставшиеся элементы докидываем в tmp,  
// и после этого (!) увеличиваем счетчики  
while (i <= mid) tmp[k++] = a[i++];  
while (j < right) tmp[k++] = a[j++];  
  
for (int k = 0; k < size; k++) {  
    a[left + k] = tmp[k];  
}  
  
delete[] tmp;  
}  
  
// Непосредственно функция сортировки  
void mergeSort(int a[], int left, int right) {  
    if (left < right) {  
        // находим середину  
        int mid = left + (right - left) / 2;  
  
        // делим пополам  
        mergeSort(a, left, mid);  
        mergeSort(a, mid + 1, right);  
  
        // сливаем результаты  
        merge(a, left, mid, right);  
    }  
}
```

Имеет сложность $O(n \log(n))$

(Дополнительно)

Оценка алгоритмов

Временная сложность - время выполнения; количество элементарных операций, необходимых для выполнения алгоритма (учитывается, как растет время выполнения, при увеличении входных данных)

Пространственная сложность - объем памяти, который требуется алгоритму для выполнения

Среди прочих критериев

- Устойчивость
- Корректность
- Простота реализации
- Универсальность